

Badr Al Jerf

Event Booking · Weekly Mini Project 02 · Team 4 **Frontend**

Responsibility: Event Details and Booking Form (Flow B and Flow C)

You own the path from choosing an event to submitting a booking. The backend is the real authority on every rule, so your job is to present the numbers clearly, guide the user, and show the backend's rejection messages exactly when something is not allowed. The project must be finished in two days, so build the core requirements first and touch any bonus only after everything else works.

You provide

- The Event Details page and the Booking Form
- The successful booking result handed to Allaith's Confirmation screen

You depend on

- Allaith: the small skeleton and the agreed way of passing the booking to the Confirmation screen (you start with your own folder and mocks)
- Majed: GET `/events/:id`
- Adham: POST `/bookings` (you start with mocks)

Working rule: all business rules are enforced in the backend. If something is unclear or the contract needs a change, tell the team leader (Allaith) so every file stays consistent.

Your Flow, Phase by Phase

Phase 1: Start Immediately with Mocks

Plan the screens and agree on the hand-off, without waiting for anyone.

- ☐ Read `API_CONTRACT.md` fully. Build only to the shapes in it.
- ☐ Allaith pushes a small skeleton in his first hours (router, empty pages, types, `StatusBadge`, `ErrorMessage`, `Loading`). Do not wait: start on your own screen in your own folder with mock data, and use the shared pieces as soon as they are pushed.
- ☐ Keep your mock data in your own file (for example `src/mocks/`), exactly matching the types in the contract.
- ☐ Own your api file (listed in your responsibilities) with two versions of each function, one mock and one real. A single switch in `.env` (`VITE_USE MOCK=true`) selects which is used.
- ☐ Read sections 6, 8 and 9 (Flow B and Flow C) of the specification.
- ☐ Your api file: `src/api/bookingApi.ts` (event by id and create booking). The mock create-booking function should also simulate rejections (Full, not enough places, duplicate) so you can test your error display.
- ☐ Agree with Allaith how a successful booking reaches the Confirmation screen (for example router state) and what happens if that screen is refreshed.

Phase 2: Event Details Screen

Show everything about one event.

- ☐ Route `/events/:id`; fetch with GET `/events/:id`.
- ☐ Show: title, description, date, time, location and category.
- ☐ Availability summary block: capacity, booked, remaining, booking percentage and status badge. Show the numbers exactly as the backend returns them.
- ☐ If the event does not exist, show "Event not found" with a link back to the Events screen.
- ☐ Loading and error states; never a blank or frozen screen.

Phase 3: Booking Entry Point

Offer booking only when it is allowed.

- ☐ Show a "Book places" button only when the event is not Full and not Past.
- ☐ When Full or Past, hide or disable it and explain why, for example "This event is full" or "This event has already started".
- ☐ Remember: this is only guidance. The backend still rejects invalid bookings from outdated pages.

Phase 4: Booking Form

Collect the booking safely.

- ☐ The form shows the selected event (title, date, time) and its current remaining places.
- ☐ Fields: full name, phone number, number of places.
- ☐ Frontend checks for a better experience: name required; phone required and plausible; places is a whole number between 1 and 4 and not more than the remaining places. Show an inline message under each field.
- ☐ On submit call `POST /bookings`. Disable the submit button while sending, so a double click cannot send twice.
- ☐ If the backend rejects the request, show its message exactly as returned in a clearly visible alert near the form, and keep everything the user typed.
- ☐ On success go to Allaith's Confirmation screen with the created booking.

Phase 5: Edge Cases

Cover the situations the specification warns about.

- ☐ Outdated page: open the form, let another booking fill the event, then submit. The backend rejection must appear clearly, and the user should be able to refresh the remaining places.
- ☐ Repeated submission and duplicate booking (same phone, same event): the backend message "You already have an active booking for this event" must be shown.
- ☐ Page refresh on the form and on the details screen must still work (data is re-fetched).
- ☐ Flow C: with an event that has few places left, book until it reaches Full and 0 remaining; further attempts must show a clear rejection, and the details page must show Full.

Phase 6: Switch from Mock to the Real Backend

When the backend owner announces an endpoint is ready, connect your screen to it.

- ☐ Set `VITE_USE MOCK=false` (or switch just your own api function) and run your screen against the real backend.
- ☐ Fix any difference between your mock and the real response, then tell the owner if the backend deviates from `API_CONTRACT.md`.
- ☐ Re-test every state on the screen with real data: success, loading, error and empty.

Phase 7: Bonus, Testing and Presentation

Only after everything above works.

- ☐ Bonus: a simple progress bar showing the booking percentage, on Details and optionally on cards (coordinate with Taima).
- ☐ Walk through Flow B and Flow C completely and try each invalid input from the specification: places 0, 5, 2.5, empty name, bad phone.
- ☐ Presentation topic: product walkthrough of Details and Booking, and the live demo of Flow B. Prepare it in English and practice it.

Definition of Done

- ☐ Details show all information and the full availability summary
- ☐ Booking button only appears when booking is allowed
- ☐ Form validates, disables while sending, shows backend messages exactly
- ☐ Successful booking reaches the Confirmation screen with correct data
- ☐ Full event shows Full and 0 remaining and rejects further attempts clearly

Your Part in the Final English Presentation

Product walkthrough of Event Details and Booking, plus the live demo of Flow B.

Appendix: API Contract Reference

Full JSON examples and every error message are in `API_CONTRACT.md`. That file is the source of truth; this table is only a summary.

Route	Input	Success	Owner
GET /events	query: category, date (YYYY-MM-DD), availableOnly=true, includePast=true	200 + Event[]	Majed
GET /events/categories	none	200 + string[]	Majed
GET /events/:id	none	200 + Event	Majed
POST /bookings	body: eventId, customerName, customerPhone, places	201 + Booking	Adham
POST /bookings/find	body: phone, code	200 + Booking	Adham
POST /bookings/:id/cancel	body: phone	200 + Booking	Adham

Event: id, title, description, date (YYYY-MM-DD), time (HH:mm), location, category, capacity, booked, remaining, bookingPercentage, status

status: "Available" | "Almost full" | "Full" | "Past"

Booking: id, code (EV-XXXX), eventId, event (full Event, with updated availability), customerName, customerPhone, places, bookedAt, status ("Active" | "Cancelled"), canCancel (boolean)

Error shape (every failure): { statusCode, message }, where message is one readable sentence ready to show to the user. Codes: 400 invalid input, 404 not found, 409 rule conflict (full, not enough places, duplicate, past event, already cancelled, event started).

This contract is a proposal. The team leader confirms the final version; if a field name changes, the leader updates this reference.